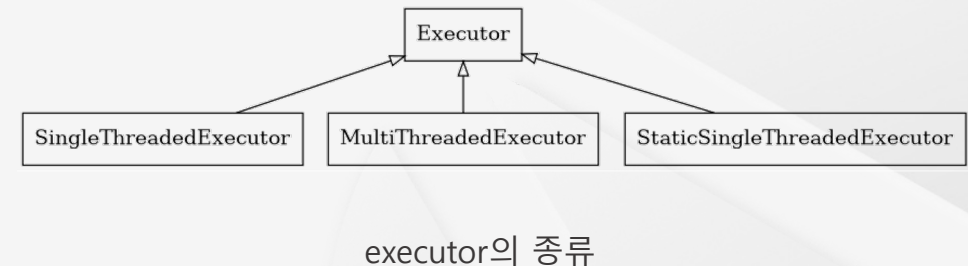
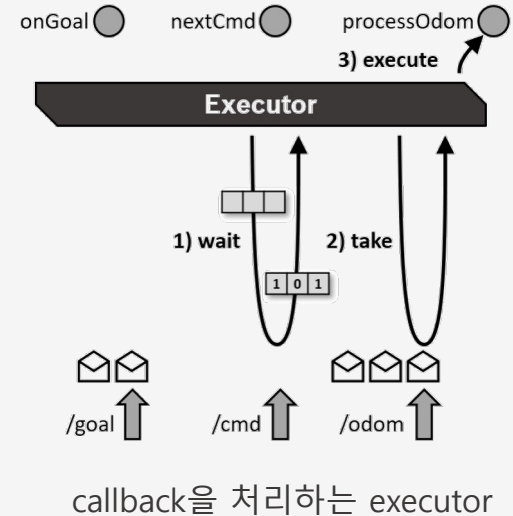


Executor & Callback Group

Executor

Executor이란?

- 노드의 콜백을 실행하는 메인 루프 역할
 - ✓ 즉, 노드의 모든 콜백(토픽 구독, 서비스, 액션, 타이머 등)을 실행하고 스케줄링
- **콜백(callback):** 특정 이벤트가 발생했을 때 실행되도록 미리 정해놓은 함수
 - ✓ 함수가 직접 호출되지 않고, 어떤 조건이 만족될 때 시스템이나 프레임워크에 의해 자동으로 호출
 - ✓ subscriber callback, timer callback, service callback 등과 같은 작업 단위
- 노드에서 발생하는 여러 이벤트(다양한 유형의 콜백)를 순차적 혹은 병렬로 처리하여 자원 활용을 최적화



Executor의 종류

- **Single-Threaded Executor**
 - ✓ 단일 스레드에서 모든 콜백을 순차적으로 처리 (병목 현상 가능성 있음)
 - ✓ 가장 단순하고 예측 가능한 실행 방식 (개발과 디버깅에 유리)
 - ✓ 동시성이 필요 없는 간단한 노드에 적합
 - ✓ 긴 시간 동안 실행되는 콜백이 있을 경우, 다른 콜백의 처리가 지연될 수 있음
- **Multi-Threaded Executor**
 - ✓ 여러 스레드를 사용하여 콜백을 병렬로 처리
 - ✓ 높은 처리량이 필요한 경우 유용
 - ✓ 복잡한 시스템에서 성능 향상 가능
 - ✓ 멀티스레드 환경에서는 스레드 간 충돌이나 경쟁 상태가 발생할 수 있어, 올바른 동기화 메커니즘을 사용하는 것이 중요

Tip) 사용 가능한 Thread 개수 확인 방법

```
> python3
Python 3.10.12 (main, Sep 11 2024, 15:47:36) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import psutil
>>> psutil.cpu_count(logical=False) # 물리적 CPU 코어 수
24
>>> psutil.cpu_count(logical=True) # 논리적 CPU 코어 수 (thread 개수)
32
```

CPU Core 개수를 간단히 확인하는 방법

```
htop 77x22

 0[1.]  4[2.]  8[2.] 12[1.] 16[0.] 20[0.] 24[0.] 28[0.]
 1[0.]  5[0.]  9[0.] 13[0.] 17[0.] 21[0.] 25[0.] 29[0.]
 2[1.]  6[0.] 10[1.] 14[0.] 18[0.] 22[0.] 26[0.] 30[0.]
 3[0.]  7[0.] 11[2.] 15[0.] 19[0.] 23[0.] 27[0.] 31[0.]
Mem[|||||] 2.82G/31.2G Tasks: 159, 775 thr; 1 running
Swp[      ] 0K/7.54G  Load average: 0.54 0.63 0.38
                               Uptime: 00:15:16
```

htop 결과

극강의 한계를 뛰어넘는 혁신적인 기술

인텔® 코어™ i9-13900KF 프로세서

24 Cores 8개 성능 코어 (P 코어) 16개 효율적인 코어 (E 코어)	32 Threads 16개 (성능 코어 기준) 16개 (효율적인 코어 기준)	5.8GHz 최대 클럭	3.0GHz 기본 클럭
36MB 인텔® 스마트 캐시	125W PBP	DDR5-5600 DDR4-3200 메모리 컨트롤러	미지원 프로세서 그래픽

i9-13900KF 사양

Callback Group

Callback Group이란?

- 콜백들의 실행 방식을 관리하고 제어하는 메커니즘
- 콜백의 실행 방식을 세분화하여 Executor가 보다 효과적으로 콜백을 관리할 수 있도록 돕는 기능
- 콜백의 실행 순서와 동시성을 관리하여, 다중 스레드 환경에서 안정적으로 노드를 운영할 수 있음
- Callback Group의 종류
 - ✓ **Reentrant Callback Group**
 - ✓ **Mutually Exclusive Callback Group**

Callback Group의 종류

- **Reentrant Callback Group**

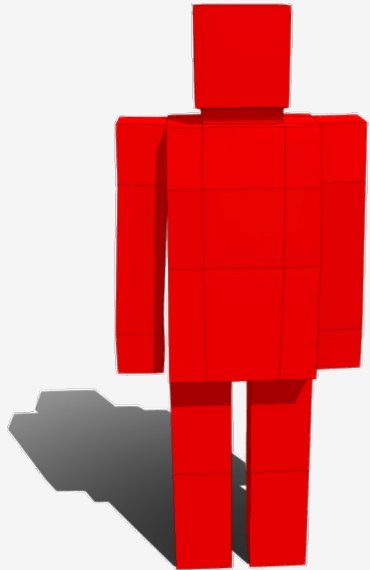
- ✓ 이 그룹으로 묶인 모든 콜백을 동시에 실행할 수 있도록 허용
- ✓ 주기적인 센서 데이터 처리나 비동기 처리를 효율적으로 할 수 있음

- **Mutually Exclusive Callback Group**

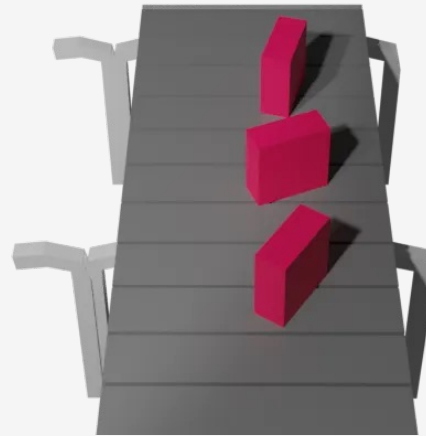
- ✓ 같은 그룹에 속한 콜백은 동시에 실행되지 않도록 보장
- ✓ 이 그룹으로 묶인 콜백들은 한 번에 하나의 콜백만 실행됨
- ✓ 특정 리소스를 보호하거나, 일부 작업이 연속적으로 실행되기를 원할 때 유용
- ✓ 데이터의 무결성을 보장해야 할 때 유용하며, 예기치 않은 동시 접근에 의한 충돌을 방지할 수 있음

Executor와 Callback Group의 이해

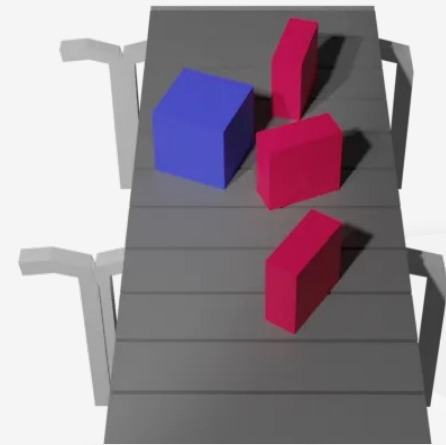
상황으로 이해하는 Executor와 Callback Group



작업자 (Executor)

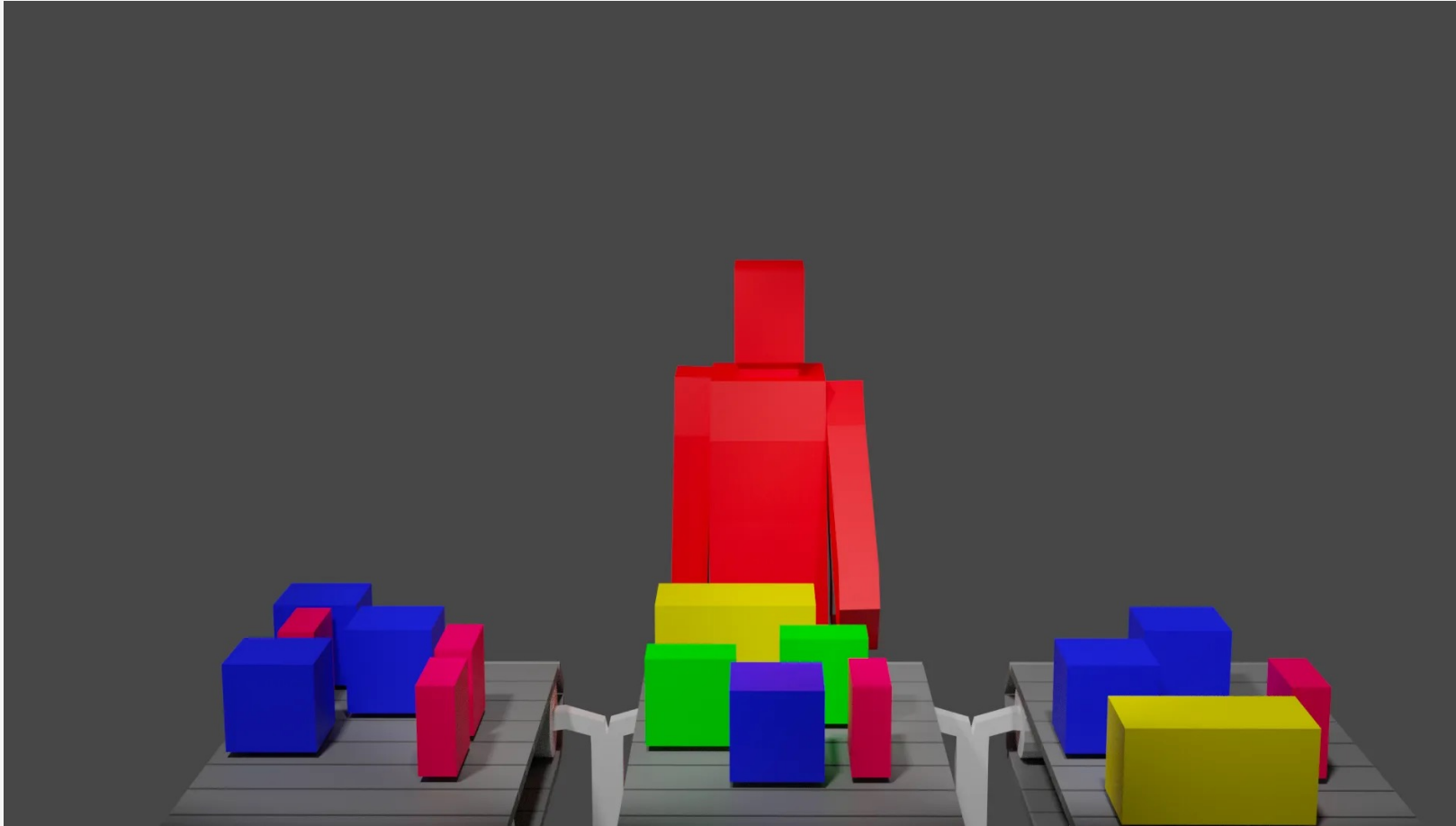


하나의 컨베이어 벨트(Callback Group)에
한 종류의 작업(Callback)이 밀려옴

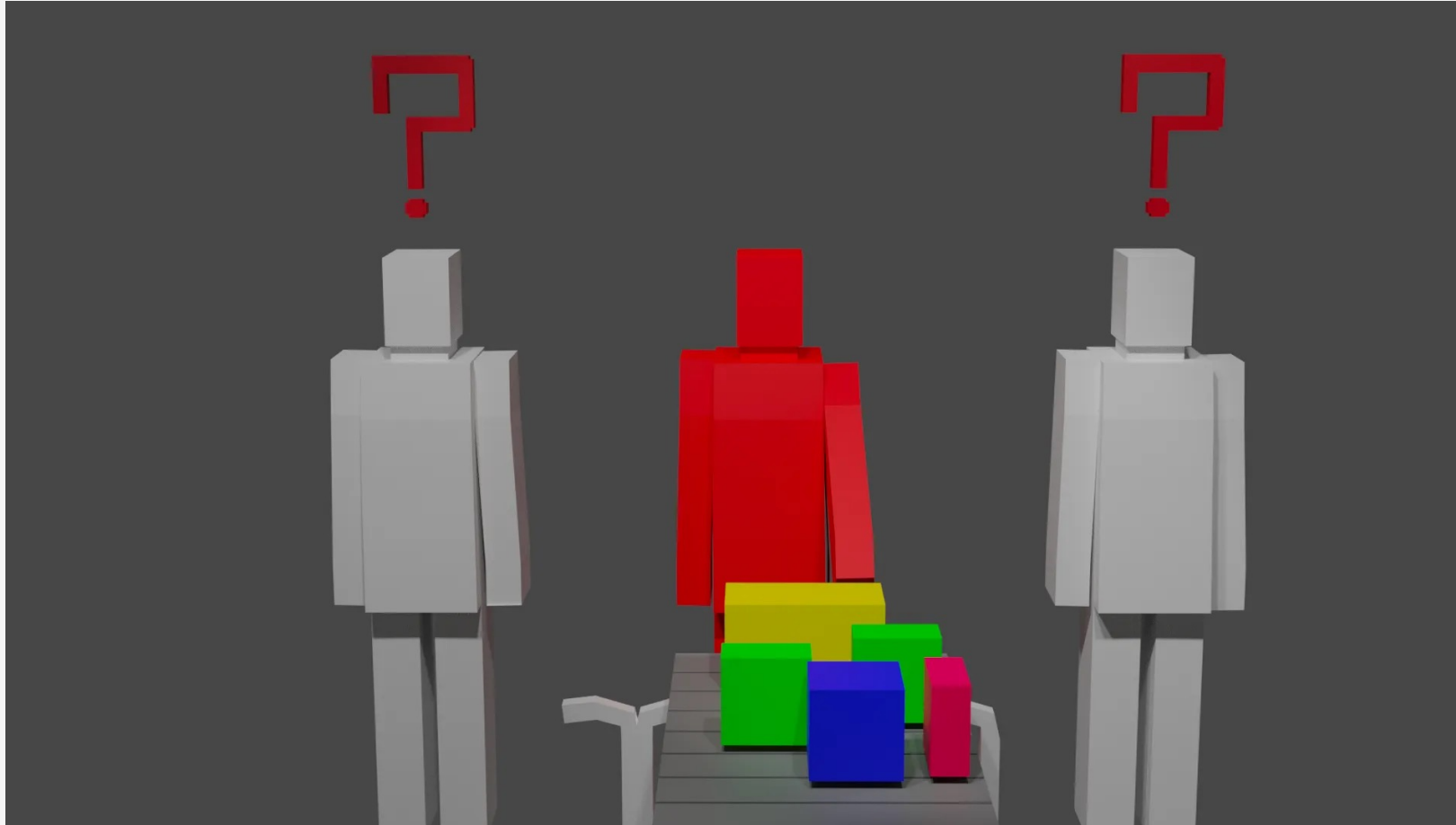


하나의 컨베이어 벨트(Callback Group)에
두 종류의 작업(Callback)들이 밀려옴

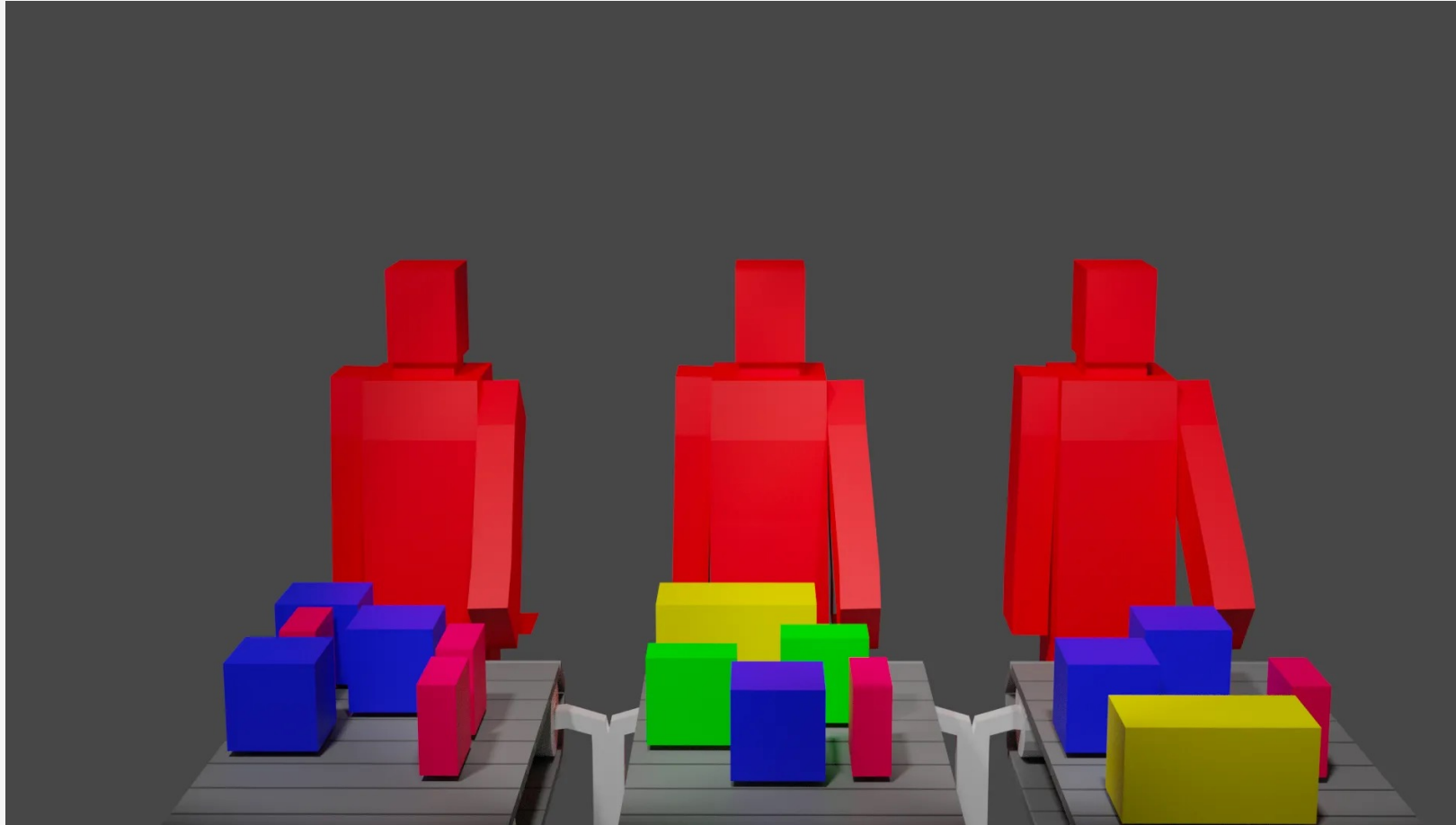
상황1) SingleThreadedExecutor와 별도의 Callback Group인 경우



상황2) MultiThreadedExecutor와 하나의 Callback Group 경우



상황3) MultiThreadedExecutor와 별도의 Callback Group인 경우



Thank You